

TUGAS PRAKTIKUM 11 ALGORITMA STRUKTUR DATA

ADJACENCY MATRIX & LIST

Nama : Alberto Relva Ezekiel Garnedy

NIM : J0403251090

1. Source Code

```
# NAMA : Alberto Relva Ezekiel Garnedy
# NIM : J0403251090
# KELAS: TPL A1

# PRAKTIKUM 1 - Membuat Adjacency Matrix
matrix = [
    [0, 1, 1, 0], # 0 terhubung dengan 1, 2
    [1, 0, 1, 0], # 1 terhubung dengan 0, 2
    [1, 1, 0, 1], # 2 terhubung dengan 0, 1
    [0, 0, 1, 0] # 3 terhubung dengan 2
]

def createMatrix(V, edges):
    # Membuat matrix V x V berisi 0
    matrix = [[0 for _ in range(V)] for _ in range(V)]

    # Menambahkan edge
    for edge in edges:
        u = edge[0]
        v = edge[1]
        matrix[u][v] = 1
        matrix[v][u] = 1
    return matrix

if __name__ == "__main__":
    V = 4
    # daftar edge
    edges = [[0, 1], [0, 2], [1, 2], [2, 3]]

    # membuat adjacency matrix
    matrix = createMatrix(V, edges)
    print("PRAKTIKUM 1: Adjacency Matrix Representation:")

    for row in matrix:
        print(row)
```

```

#PRAKTIKUM 2 - Membuat Adjacency List
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"]
}

def createGraph(V, edges):
    adj = [[] for _ in range(V)]
    # Add each edge to adjacency list
    for it in edges:
        u = it[0]
        v = it[1]
        adj[u].append(v)
        adj[v].append(u) # karena undirected graph
    return adj

if __name__ == "__main__":
    V = 4
    # List of edges (u, v)
    edges = [[0,1], [0,2], [1,3], [2,3]]

    # Build the graph using edges
    adj = createGraph(V, edges)
    print("PRAKTIKUM 2: Adjacency List Representation:")

    for i in range(V):
        # Print the vertex
        print(f"{i}:", end=" ")

        for j in adj[i]:
            # Print its adjacent
            print(j, end=" ")
        print()

# PRAKTIKUM 3 - Konversi Matrix ke List
matrix = [ [0,1,1,0], [1,0,1,0], [1,1,0,1], [0,0,1,0] ]
list = {
    0 : [1, 2],
    1 : [0, 2],
    2 : [0, 1, 3],
    3 : [2]
}

```

```

# PRAKTIKUM 4 - Studi Kasus Dunia Nyata (Digit Akhir NIM: 6)
# Langkah 1: Digit akhir = 6 (Studi kasus = Peta Kota)

# Langkah 2: Tentukan Node dan Edge
# node = ["Bogor", "Jakarta Barat", "Jakarta Selatan", "Senen", "Tangerang"]
# Edge = Jalan penghubung

# Langkah 3: Gambar Desain Graph
# Langkah 3_Praktikum 4_P11.drawio.png

# Langkah 4: Implementasi dalam Python
graph = {
    "Jakarta Selatan": ["Jakarta Barat", "Jakarta Pusat", "Tangerang", "Bogor"],
    "Jakarta Barat": ["Jakarta Selatan", "Jakarta Pusat", "Tangerang"],
    "Jakarta Pusat": ["Jakarta Selatan", "Jakarta Barat"],
    "Tangerang": ["Jakarta Selatan", "Jakarta Barat"],
    "Bogor": ["Jakarta Selatan"]
}

matrix = [
    [0, 1, 1, 1, 1],
    [1, 0, 1, 1, 0],
    [1, 1, 0, 0, 0],
    [1, 1, 0, 0, 0],
    [1, 0, 0, 0, 0]
]

nodes = [
    "Jakarta Selatan",
    "Jakarta Barat",
    "Jakarta Pusat",
    "Tangerang",
    "Bogor"
]

# LANGKAH 5 - Tampilkan Output Program

def createGraphs(edges):
    graph = {}
    # Add each edge to adjacency list
    for u, v in edges:
        # jika node belum ada
        if u not in graph:
            graph[u] = []

        if v not in graph:

```

```

        graph[v] = []
        # karena undirected graph
        graph[u].append(v)
        graph[v].append(u)
    return graph

if __name__ == "__main__":
    # List of edges (u, v)
    edges = [
        ["Jakarta Selatan", "Jakarta Barat"],
        ["Jakarta Selatan", "Jakarta Pusat"],
        ["Jakarta Selatan", "Tangerang"],
        ["Jakarta Selatan", "Bogor"],
        ["Jakarta Barat", "Jakarta Pusat"],
        ["Jakarta Barat", "Tangerang"]
    ]

    # Build the graph using edges
    adj = createGraphs(edges)
    print("PRAKTIKUM 4 Langkah 5.1: Adjacency List Representation:")

    for node in adj:
        # Print the vertex
        print(f"{node}:", end=" ")

        # Print its adjacent
        for neighbor in adj[node]:
            print(neighbor, end=" ")
        print()

    print("PRAKTIKUM 4 Langkah 5.2: Adjacency Matrix Representation:")
    for row in matrix:
        print(row)

    print("PRAKTIKUM 4 Langkah 5.3: Nama Node")
    for node in nodes:
        print("-", node)

    print("PRAKTIKUM 4 Langkah 5.4: Hubungan Antar Node")
    for node in graph:
        for neighbor in graph[node]:
            print(node, "<->", neighbor)

```

2. Output

PRAKTIKUM 1: Adjacency Matrix Representation:

[0, 1, 1, 0]

[1, 0, 1, 0]

[1, 1, 0, 1]

[0, 0, 1, 0]

PRAKTIKUM 2: Adjacency List Representation:

0: 1 2

1: 0 3

2: 0 3

3: 1 2

PRAKTIKUM 4 Langkah 5.1: Adjacency List Representation:

Jakarta Selatan: Jakarta Barat Jakarta Pusat Tangerang Bogor

Jakarta Barat: Jakarta Selatan Jakarta Pusat Tangerang

Jakarta Pusat: Jakarta Selatan Jakarta Barat

Tangerang: Jakarta Selatan Jakarta Barat

Bogor: Jakarta Selatan

PRAKTIKUM 4 Langkah 5.2: Adjacency Matrix Representation:

[0, 1, 1, 1, 1]

[1, 0, 1, 1, 0]

[1, 1, 0, 0, 0]

[1, 1, 0, 0, 0]

[1, 0, 0, 0, 0]

PRAKTIKUM 4 Langkah 5.3: Nama Node

- Jakarta Selatan

- Jakarta Barat

- Jakarta Pusat

- Tangerang

- Bogor

PRAKTIKUM 4 Langkah 5.4: Hubungan Antar Node

Jakarta Selatan <-> Jakarta Barat

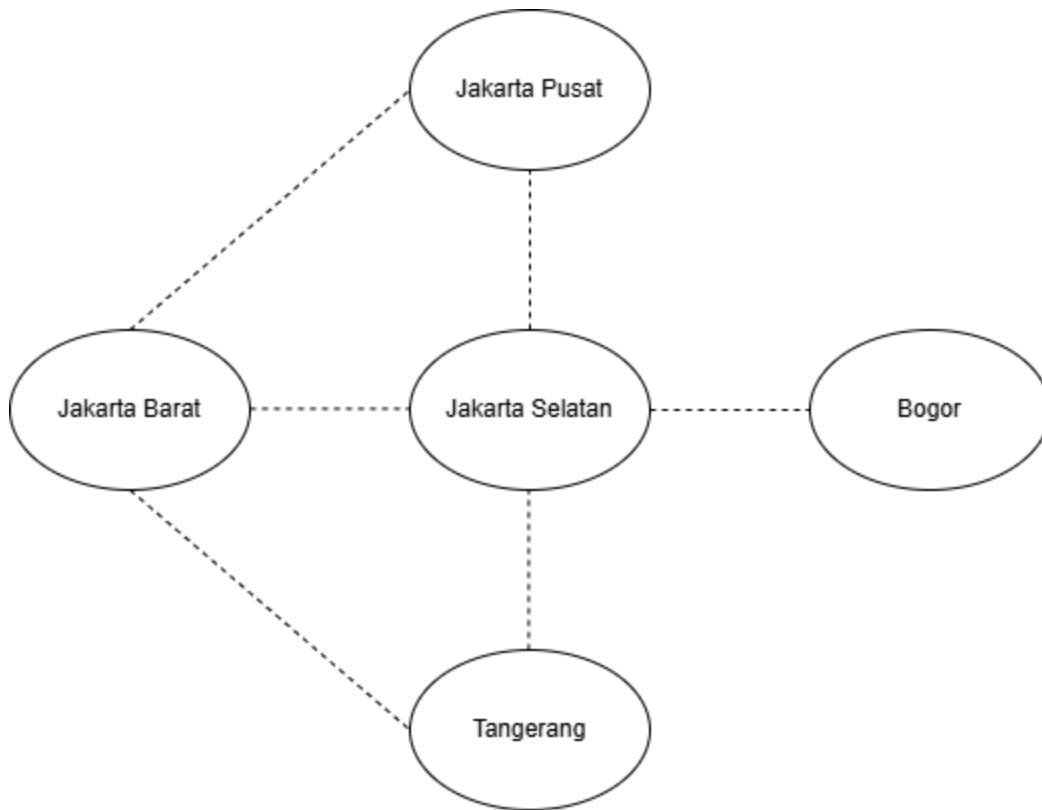
Jakarta Selatan <-> Jakarta Pusat

Jakarta Selatan <-> Tangerang

Jakarta Selatan <-> Bogor

Jakarta Barat <-> Jakarta Selatan

3. Grafik



4. Penjelasan Node dan Edge

Node nya ada 6 yaitu Bogor, Jakarta Barat, Jakarta Pusat, Tangerang, Bogor Edgenya garis penghubung antar kota:

- Jakarta Selatan → Jakarta Barat
- Jakarta Selatan → Jakarta Pusat
- Jakarta Selatan → Tangerang
- Jakarta Selatan → Bogor
- Jakarta Barat → Jakarta Pusat
- Jakarta Barat → Tangerang

5. Analisis adjacency list vs matrix

Adjacency list menyimpan daftar tetangga dari setiap node (cocok untuk graph besar dan mudah melihat node yang saling terhubung. Sedangkan Adjacency matrix menyimpan hubungan graph dalam bentuk tabel 2 dimensi (pengecekan hubungan antar node sangat cepat dan cocok untuk graph dengan banyak edge (dense graph)).

6. Kesimpulan singkat

Pada Praktikum 4, graph digunakan untuk merepresentasikan hubungan antar kota pada studi kasus peta kota. Setiap kota direpresentasikan sebagai node, sedangkan jalan penghubung direpresentasikan sebagai edge. Graph yang dibuat termasuk undirected

graph karena hubungan antar kota dapat dilalui dua arah. Representasi graph diimplementasikan menggunakan adjacency list dan adjacency matrix dalam Python.